1. Mục tiêu

- Hiểu rõ tư tưởng và tầm quan trọng của các nguyên lý SOLID trong thiết kế phần mềm.
- Nắm vững và có khả năng áp dụng nguyên lý **Single Responsibility Principle (SRP)** và **Open/Closed Principle (OCP)** vào thực tế.
- Chuẩn bị tốt cho phần thuyết trình và demo.

2. Nội dung cần nghiên cứu và chuẩn bị

2.1. Single Responsibility Principle (SRP)

- **Khái niệm:** Một class (hoặc module, function) chỉ nên có **một và chỉ một** lý do để thay đổi. Điều này có nghĩa là mỗi class chỉ nên chịu trách nhiệm cho một phần chức năng cụ thể.
- **Tầm quan trọng:** Giúp code dễ đọc, dễ bảo trì, dễ mở rộng và giảm thiểu lỗi.
- **Ví dụ:**
  - **Không áp dụng SRP:** Một class `User` vừa quản lý thông tin người dùng, vừa xử lý lưu trữ vào database, vừa gửi email thông báo.
  - **Áp dụng SRP:** Tách class `User` thành `User`, `UserRepository` (quản lý database), và `EmailService` (gửi email).
- **Chuẩn bị demo:** Viết một ví dụ code (có thể là C#, Java, Go) minh họa rõ ràng cả trường hợp vi phạm và áp dụng SRP. Giải thích tại sao việc áp dụng SRP lại tốt hơn.

2.2. Open/Closed Principle (OCP)

- **Khái niệm:** Một thực thể phần mềm (class, module, function, v.v.) nên **mở để mở rộng, nhưng đóng để sửa đổi**. Điều này có nghĩa là bạn có thể thêm chức năng mới mà không cần thay đổi code hiện có.
- **Tầm quan trọng:** Giúp hệ thống linh hoạt, dễ dàng thêm tính năng mới mà không ảnh hưởng đến các phần đã hoạt động ổn định.
- **Ví dụ:**
  - **Không áp dụng OCP:** Một hàm tính toán lương dựa trên loại nhân viên sử dụng nhiều câu lệnh `if-else` hoặc `switch-case` để xử lý từng loại nhân viên. Khi thêm loại nhân viên mới, phải sửa đổi hàm này.
  - **Áp dụng OCP:** Sử dụng Interface hoặc Abstract Class và kế thừa (Polymorphism) để định nghĩa hành vi chung. Mỗi loại nhân viên mới sẽ là một class riêng kế thừa từ Interface/Abstract Class đó, không cần sửa đổi hàm tính toán lương gốc.
- **Chuẩn bị demo:** Viết một ví dụ code (có thể là C#, Java, Go) minh họa rõ ràng cả trường hợp vi phạm và áp dụng OCP. Giải thích cách OCP giúp mở rộng hệ thống dễ dàng hơn.

3. Yêu cầu Framework

- **Bắt buộc:** Sử dụng framework có ràng buộc kiểu dữ liệu tĩnh như C# ASP.NET Core, Java Spring Boot, Go, v.v. để minh họa các nguyên lý này một cách rõ ràng nhất.

4. Chuẩn bị thuyết trình và demo

- **Slide:** Chuẩn bị slide thuyết trình ngắn gọn, tập trung vào các khái niệm chính, tầm quan trọng và ví dụ minh họa.

- **Code Demo:** Đảm bảo code demo chạy được, dễ hiểu và minh họa đúng nguyên lý. Chuẩn bị sẵn các kịch bản demo (ví dụ: chạy code vi phạm nguyên lý, sau đó chạy code đã refactor theo nguyên lý).
- **Thời gian:** Phân bổ thời gian hợp lý cho từng phần (lý thuyết, ví dụ, demo).

5. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu sâu về SRP và OCP	TV1	2
Chuẩn bị nội dung slide thuyết trình	TV2	2
Viết code demo SRP (có/không áp dụng)	TV3	3
Viết code demo OCP (có/không áp dụng)	Nhóm trưởng	3
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**13**

6. Tài liệu tham khảo

- **Sách:** "Clean Code" của Robert C. Martin (Uncle Bob) - Chương về SOLID Principles.
- **Bài viết:** Tìm kiếm các bài viết về SOLID principles trên các trang web như Medium, Dev.to, hoặc blog của các lập trình viên kinh nghiệm.
- **Video:** Xem các video hướng dẫn trên YouTube về SOLID principles, đặc biệt là SRP và OCP.
- **Tài liệu chính thức:** Đọc tài liệu của framework mà nhóm chọn sử dụng (ASP.NET Core, Spring Boot, v.v.) để hiểu cách áp dụng các nguyên lý này trong thực tế.

7. Ghi chú quan trọng

- **Thực hành là chìa khóa:** Không chỉ đọc lý thuyết mà phải thực sự viết code và chạy thử để hiểu sâu hơn.
- **Ví dụ thực tế:** Cố gắng tìm hoặc tạo ra các ví dụ gần gũi với thực tế, không quá trừu tượng.
- **Chuẩn bị câu hỏi:** Dự đoán các câu hỏi có thể được hỏi trong buổi thuyết trình và chuẩn bị câu trả lời.
- **Backup plan:** Chuẩn bị sẵn phương án dự phòng trong trường hợp demo gặp sự cố kỹ thuật.

Chúc các bạn học tập hiệu quả và thành công!